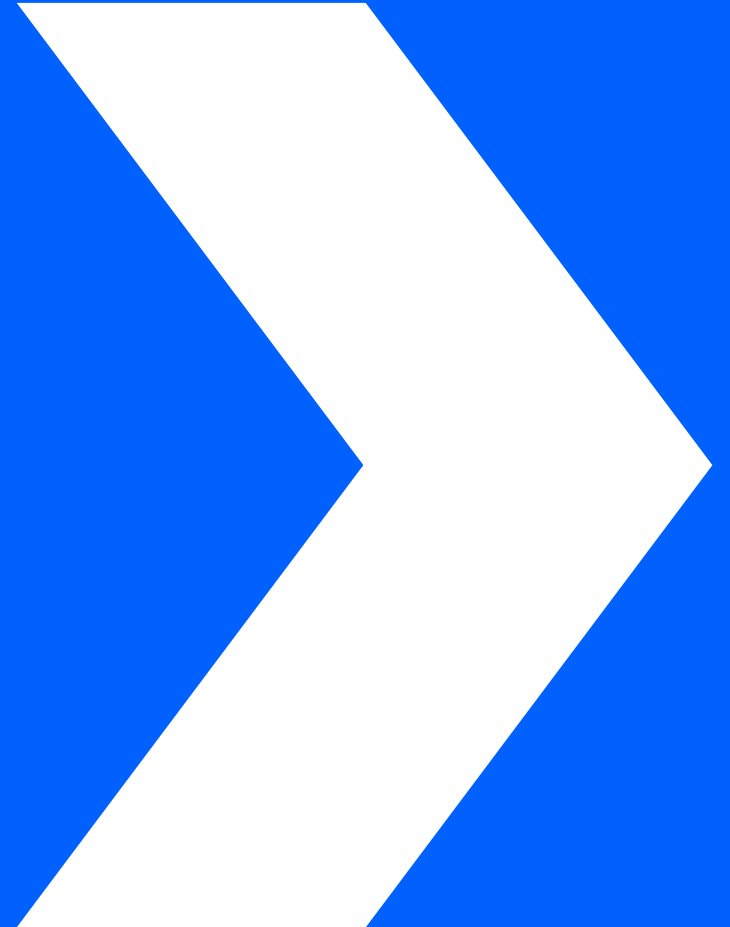


Creating an Autonomous LLM Research Agent

NLP with Disaster Tweets



Simon Gühring, Rahul Rajesh, Niccolò Germani & Dominik Plöchl

Dataset & Task

The Dataset

~7,600

labeled training tweets

~3,200

unlabeled test tweets

57%

not-disaster class (majority)

43%

actual disaster class

The Goal

Maximize F1 score on the held-out test set via an autonomous agent that runs, evaluates, and iterates models without human input.

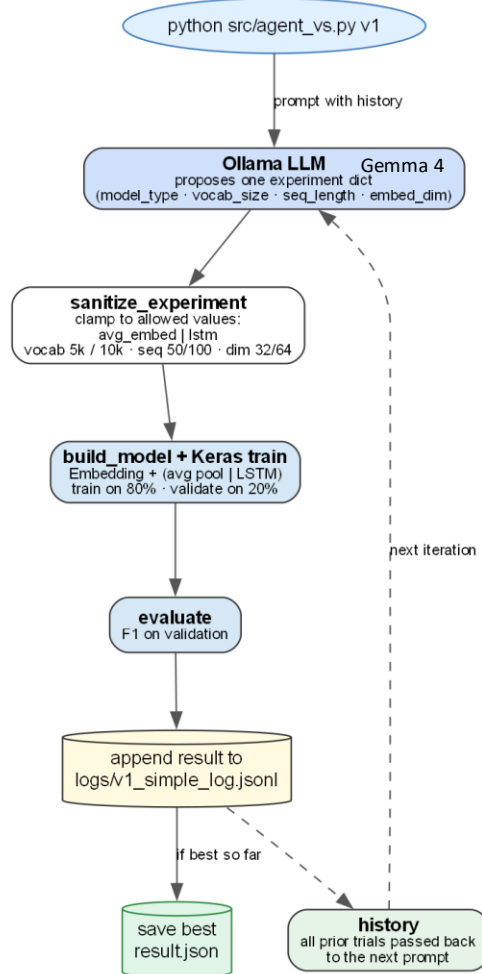
The Task



Given a tweet, predict whether it describes a real disaster (1) or is unrelated (0).

V1 – First Attempt single Autonomous Agent

V1 — Single-architecture LLM Baseline Agent



Main Challenges

1

Problems loading data

FileNotFoundError

2

Buggy generated code

Function declares parameter X_va but reads X_va_sparse from outer scope

3

Families mostly hardcoded

LLM only proposes a config DICT. The training script is hardcoded.

4

No repair, no preflight

A bad spec or runtime crash kills the trial outright.

5

LLM didn't follow the spec

Prompt asked for LogisticRegression(C=1.0, ...); the code uses LogisticRegressionCV(..., cv=5)

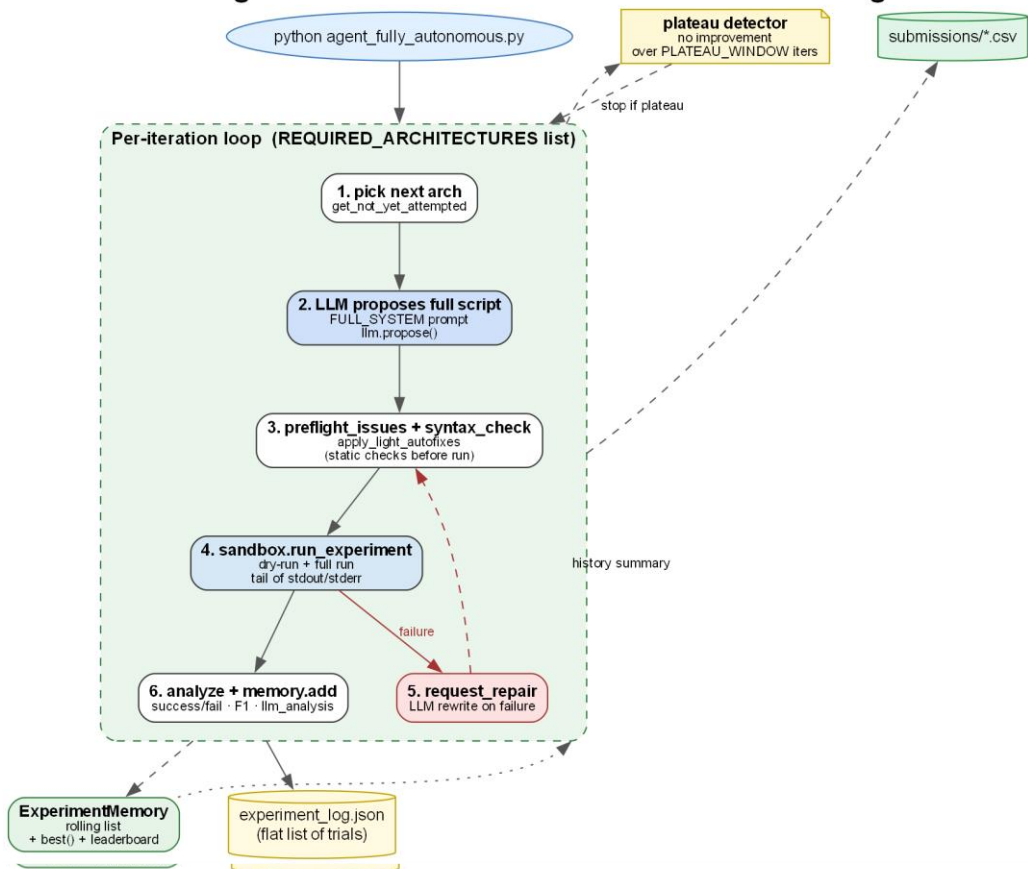
6

Off-by-axis indexing in the submission write.

test_probs[va_idx] uses training-row indices on a test-shaped array.

V2 - Single-file Autonomous Multi-architecture Agent

V2 — Single-file Autonomous Multi-architecture LLM Agent



Families tested

BoW | BoW_advanced | CNN | LSTM | Transformer

Best local F1
0.8209 (Transformer)

Kaggle F1
0.8284

Improvements over V1

- 1 LLM writes the full script**
FULL_SYSTEM prompt - LLM now owns the complete training script, not just a config dict.
- 2 Preflight + autofix before execution**
preflight_issues + syntax_check + apply_light_autofixes before every sandbox call.
- 3 Repair loop on failure**
sandbox crash → request_repair → LLM rewrites → back through preflight.
- 4 Multi-architecture sweep**
REQUIRED_ARCHITECTURES + get_not_yet_attempted - expands beyond V1's 16-combo space.
- 5 Dry run before full execution**
60 s dry run on a tiny data head catches crashes before the 1000 s full run - V1 had no such safety net.

Remaining Challenges

- 1 Plateau detector didn't fire**
BoW F1=0.7808x5 runs, LSTM F1=0.6008x5 runs. Zero movement; agent burned all attempts anyway
- 2 Fake variation across runs**
Only max_features jitters (15000 → 14800) across 5 BoW runs; every other key identical. One-axis search.
- 3 Silent degenerate results**
LSTM accuracy=0.0 on 5 runs, marked success=True. F1 alone hides one-class collapse - no degenerate detection
- 4 Analyst feedback ignored**
Analyst: "increase to 2+ LSTM layers"; next spec still has num_layers=1. History → propose loop doesn't move the spec.



Best local F1

Kaggle F1

0.842

1

Propoyser: gemma4:31b(strategy). Coder: qwen2.5-coder:14b (code generation).

2

Separate prompt template per family - BoW, CNN, LSTM, RoBERTa, BERTweet, Transformer.

3

Phase A explores all families; Phase B tunes top-2 with 10 k rows; Phase C retrain on full data.

4

LLM patches specific lines rather than regenerating the whole script - less token waste.

5

Best families get dedicated hyperparameter sweeps before the final submission run.

1

Accuracy=0.0 but no exception raised - trial marked 'success', budget wasted.

2

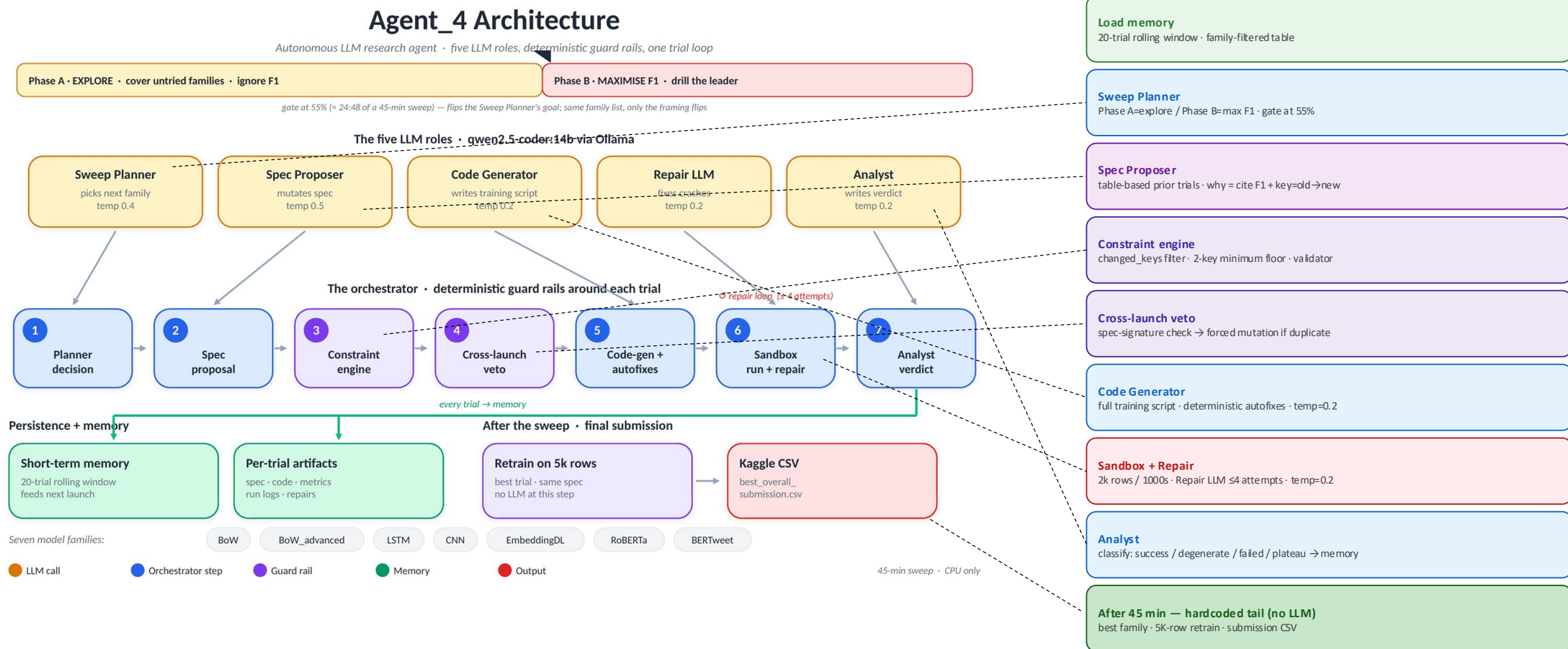
LSTM ran 3 times, F1=0.0 each time, no exceptions raised- all marked success=True.

3

Phase C had a syntax error - no CSV produced despite 296 completed sweep trials.

Agent 4 - Final Architecture

Final Architecture

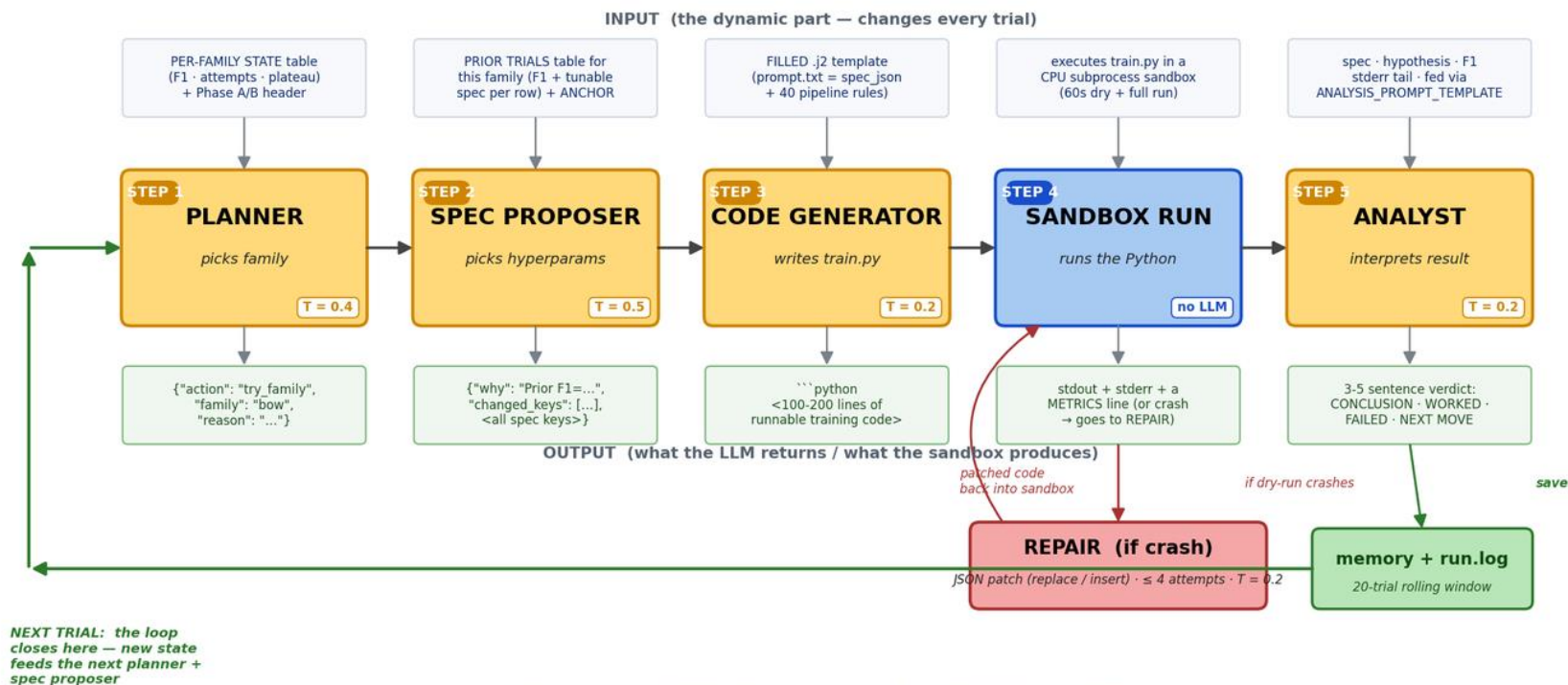


The Agent's Prompt Chain

5 LLM roles per trial

The 5-prompt chain in ONE trial

Every trial calls the LLM five times. Each call has a different role, a different system prompt, and a different temperature.



The whole loop runs once per trial · typically 8-15 trials in a 45-min sweep.

Planner = 1 system prompt + dynamic state. Spec proposer = 1 system prompt + dynamic table. Code-gen = 1 system + the filled .j2 template. Repair = 1 system + crash trace. Analyst = 1 templated prompt.

Picks WHICH model family to try next; $T = 0.4$

PHASE A — EXPLORE

```
=== CURRENT PHASE: A (EXPLORE) ===
Your ONE goal right now is to COVER
families and parameter regions you
have not yet measured this launch.
Do NOT optimise for F1 yet – that
comes in Phase B. A trial that
explores an untried family right now
is more valuable than a trial that
drills the current leader.
```

PHASE B — MAXIMISE F1

```
=== CURRENT PHASE: B (MAXIMISE F1) ===
Exploration is OVER. Your ONE goal
now is to push F1 ABOVE the current
leader: bertweet (F1=0.7812). Do NOT
pick an untried family for 'coverage'
– pick whichever family is most
likely to lift F1 ABOVE that number.
Usually this means drilling the
current leader with parameter tweaks.
```

WHAT IT SEES PRE-RUN — PER-FAMILY STATE TABLE (sweep_planner.py)

family	att	ok	bestF1	lastF1	outcome	wall	cost	eligible
bertweet	3	2	0.7812	0.7706	success	420s	480s	yes
roberta	2	1	0.7654	fail	code_gen_failed	380s	480s	yes
bow	4	4	0.7296	0.6963	success (stagnant)	45s	60s	yes
cnn	1	1	0.6890	0.6890	success	180s	180s	yes
lstm	1	0	–	–	sandbox_timed_out	600s	600s	yes
bow_advanced	0	0	–	–	–	0s	60s	yes
embedding_dl	0	0	–	–	–	0s	120s	yes

System prompt:

prompts.py:46 → SWEEP_PLANNER_SYSTEM

Phase A/B header builder:

sweep_planner.py:285 → _phase_header()

OUTPUT (one line of JSON):

```
{"action": "try_family",
 "family_key": "bertweet",
 "reason": "bertweet F1=0.7812 is
current leader; tweak lr to push
past 0.78 in Phase B"}
```

2/5 SPEC Proposer

Picks WHICH hyperparameters to try in the chosen family ; $T = 0.5$

SYSTEM PROMPT · constant rulebook

You propose ONE hyperparameter spec for the Kaggle Disaster Tweets task.

PRIMARY GOAL: pick a spec likely to beat the best F1 in the PRIOR TRIALS table shown in the user prompt.

Mutate the ANCHOR. Change AT LEAST 2 tunable keys. Skip keys flagged plateaued. Don't repeat any spec in the table. Avoid regions that crashed.

Output ONE JSON object on a single line:
 {"why": "<prior F1 + your move>",
 "changed_keys": [...], <spec keys>}

prompts.py:21 → SPEC_PROPOSER_SYSTEM

USER PROMPT · changes every turn (real BERTweet run)

Propose the next BERTweet spec.

PRIOR TRIALS for BERTweet (newest first):

F1	tunable spec	source	outcome
fail	max_len=104, lr=1.3e-5, wd=0.008	this launch	code_gen_failed
fail	max_len=80, lr=1.2e-5, wd=0.01	this launch	code_gen_failed
0.7706	max_len=104, lr=1.5e-5, wd=0.01	this launch	success
fail	max_len=104, lr=2.0e-5, wd=0.01	prior launch	code_gen_failed

ANCHOR (current best successful):

```
{"max_len":104, "lr":1.5e-5, "wd":0.01,
"batch":13, "epochs":3}
```

TUNABLE KEYS: max_len, batch_size, lr, wd, epochs

runs/<exp>/run_001/search_prompt.txt

→ OUTPUT · one JSON line returned by the LLM:

```
{"why": "Prior F1=0.7706 with lr=1.5e-5, max_len=104; changing
lr=1.5e-5→8e-6 and max_len=104→160 to test slower training +
longer context to push above 0.7706.",
"changed_keys": ["learning_rate", "max_len"],
"learning_rate": 8e-6, "max_len": 160, "train_batch_size": 13, ...}
```

Next call's PRIOR TRIALS table will include THIS new spec + its F1 — the feedback loop.

3/5 Code Generator

Turns the SPEC into a complete runnable Python script, $T=0.2$

USER PROMPT · saved as prompt.txt

```
Write one complete runnable Python script
for Kaggle Disaster Tweets.

Family: BoW
Use a simple sklearn bag-of-words baseline
with TF-IDF features and logistic regression.

Use this validated spec exactly:
{
  "architecture": "Bow",
  "vectorizer_type": "tfidf_word",
  "max_features": 25000,
  "ngram_max": 3,
  "min_df": 2,
  "logreg_c": 6.0,
  "val_size": 0.2,
  "threshold_min": 0.3,
  "threshold_max": 0.7,
  "threshold_steps": 41
}

Required pipeline:
- load train.csv, test.csv from DATA_DIR
- fillna keyword, location, text with ""
- build text = keyword + " [SEP] " + text
- train_test_split test_size=0.2, stratify=y
- TfidfVectorizer with spec values
- LogisticRegression with C from spec
- tune threshold over linspace(0.3,0.7,41)
- print final METRICS line
```

runs/bow_20260516_153039/run_001/prompt.txt



LLM
writes

LLM OUTPUT · saved as train.py

```
import os, pandas as pd, numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score

DATA_DIR = os.environ.get('DISASTER_AGENT_DATA_DIR')
train_df = pd.read_csv(f'{DATA_DIR}/train.csv')
test_df = pd.read_csv(f'{DATA_DIR}/test.csv')

for _df in (train_df, test_df):
    for c in ('keyword', 'location', 'text'):
        _df[c] = _df[c].fillna('').astype(str)

train_df['t'] = train_df['keyword'] + ' [SEP] ' + train_df['text']
test_df['t'] = test_df['keyword'] + ' [SEP] ' + test_df['text']

X_tr, X_va, y_tr, y_va = train_test_split(
    train_df['t'], train_df['target'],
    test_size=0.2, random_state=42,
    stratify=train_df['target'])

vec = TfidfVectorizer(max_features=25000,
                      ngram_range=(1,3), min_df=2)
X_tr = vec.fit_transform(X_tr)
X_va = vec.transform(X_va)

m = LogisticRegression(C=6.0,
                      random_state=42).fit(X_tr, y_tr)
p = m.predict_proba(X_va)[:,1]

best_f1, best_t = 0, 0.5
for t in np.linspace(0.3, 0.7, 41):
    f1 = f1_score(y_va, (p>=t).astype(int))
    if f1>best_f1: best_f1, best_t = f1, t
```

runs/bow_20260516_153039/run_001/train.py

4/5 + 5/5 • Repair + Analyst

If the script crashes, REPAIR patches it (≤ 4 tries). After it runs, ANALYST writes the post-mortem

REPAIR · T = 0.2 · surgical patches

When train.py crashes, the agent sends:

- the broken code
- the stderr traceback
- the diagnosis hint

LLM returns ONE JSON patch (real example):

```
{
  "diagnosis": "np is not defined",
  "edits": [
    {
      "action": "insert_before",
      "target": "for threshold in
        np.linspace(...):",
      "content": "import numpy as np"
    }
  ]
}
```

→ Up to 4 retries (MAX_REPAIR_ATTEMPTS).

→ It's a DIFF, not a rewrite.

prompts.py:14 → PATCH_REPAIR_SYSTEM

ANALYST · T = 0.2 · closes the loop

After train.py finishes, sees:

- the spec proposer's hypothesis
- the final spec JSON
- METRICS line, stdout + stderr tail

LLM writes 3–5 sentences (real example):

CONCLUSION: hypothesis was REFUTED.
Switching max_features 18k→25k and
ngram_max 2→3 gave F1=0.6963, BELOW
the prior best F1=0.7296.

WHAT WORKED: pipeline ran clean end
to end, no errors.

WHAT FAILED: larger n-grams added
noise; F1 dropped by 0.033.

NEXT MOVE: revert ngram_max=2, try
logreg_c=8 with min_df=3.

→ Verdict goes into memory + run.log.

→ Next Planner call sees it.

prompts.py:84 → ANALYSIS_PROMPT_TEMPLATE

THE FEEDBACK LOOP

Analyst's verdict → added to PRIOR TRIALS → Spec Proposer reads it next turn → better hypothesis.